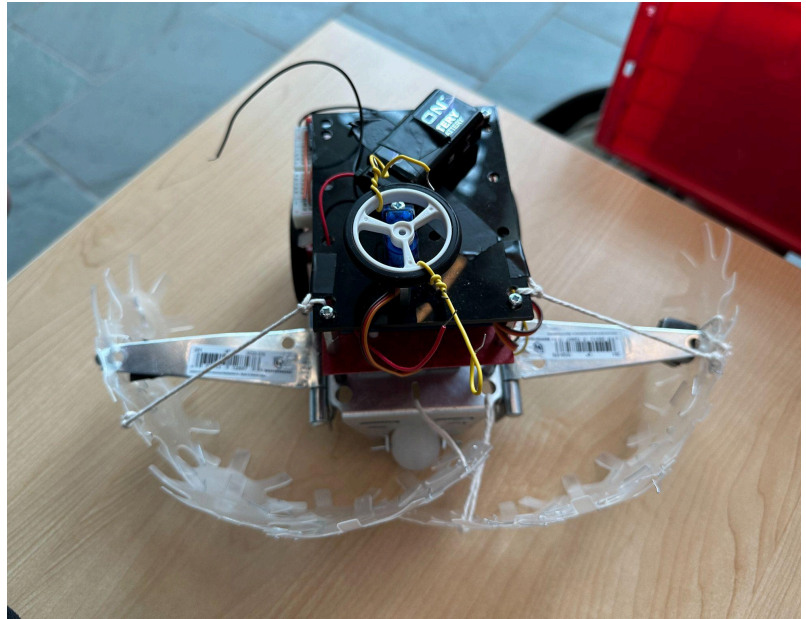


MAE 3780 Robot Competition Final Report

Group 49: David Almeida (dja233), Jeffrey Ruiz (jr996), Kate Ruelan (kr579)

Robot Design and Strategy Overview:

The robot is designed with two arms that lower at the start of the competition to create a basket-like alcove to catch the cubes. This was accomplished with two hinges mounted to the chassis of the robot that are able to drop down at the start using a makeshift hook and latch mechanism and a servo. The hinges expand the robot to the full width of the 12-inch



circumference allowed for full robot expansion to maximize area to catch cubes as it moves around the board. During the competition, the robot uses two QTI sensors mounted on the forward end to sense the black borders of the playing field to navigate within the borders.

The strategy is to have the robot perform loops around the middle of the playing field to collect cubes in the highest concentration area. We anticipate that many robots will start moving towards the center at the start of the competition, so we plan on starting from the sides of the field.

Design Process Reflection

Our focus of design was mostly on the actual mechanism that would be collecting the cubes during competition. Because of this, we did not very intentionally think about where necessary things for robot functioning should be

mounted. For the beginning milestones, we worked with the larger breadboard from the kit precariously placed on top of the smaller one mounted on the arduino board. At some point before all of the sensors were connected in the circuit, it seemed like maybe we could just use the smaller one, so we had switched all our wiring, and then ended up needing to switch it back again at some point and just decided to take the smaller breadboard off where it was mounted and put the larger one in its place. Similar things happened with battery placement, and in the end we just ended up using electrical tape to secure our 9V battery to the top of our robot. In hindsight, putting more intentional thought into where the necessary components for robot functioning would be placed would have been helpful.

One problem we encountered was that one of our micro servos was not working properly, and we had to make a change to the physical design of the robot to accommodate the use of only one micro servo. This physical change did not affect the overall performance since we planned ahead with our laser cut piece to allow several possible positions of the servo to be mounted. It did change the way we used the servo to release our arms, which due to the makeshift nature of the hook and latch release, was inconsistent at many points. Luckily, we had gotten the hang of how to secure it just right by competition day.

Competition Analysis

On competition day the robot had mixed results for several reasons. During a few of the matches our robot successfully retrieved a number of blocks that would have won the match, however our robot was either pushed off the board or had our blocks taken by the other robot. We did not anticipate that the other robot would have enough mass to push our robot across the board as we had deliberately added more weight to prevent this occurrence. Specifically, this mostly caused us to lose blocks in the situation of being pushed backward from a head-on position of the other robot. We had talked briefly about a plan to create some sort of gate mechanism during the build process to help keep the blocks in the alcove we had built to collect them, but because the other aspects of our build took longer than initially expected, this was not something we had time to complete.

The other issues we had were around false starts that would cause our robot to not execute our code properly, and remain stationary. There were several

reasons including a loose wire, and our battery dropping voltage faster than we anticipated. The competition presented a challenge of long term endurance for the battery that we had not encountered during lab testing.

Conclusions

If keeping completely with our initial design idea, the number one improvement that we should have focused on was creating a way for the blocks to be retained by our robot, as this was the number one thing that voided all of our hard work come competition day. Another thing that could have improved our robot performance was spending more time adjusting the code. Specifically, the angles of turning. This was not refined completely just due to a time crunch in the end. Also, when testing our robot, we never let it run for the full length of the match, which we should have done, because our robot did stop after about 40 seconds and we still had more time where we could have been moving around collecting blocks or moving so we were not a sitting duck for other groups robots to hit us head on. One thing that would make the mechanical design of this project easier is if we did not utilize the lower mechanical arms design that we had planned. If we were to have focused on making a rigid laser cut design in the shape of an 8"x8" box, then we could cut down on the code, power, and moving parts complexity.

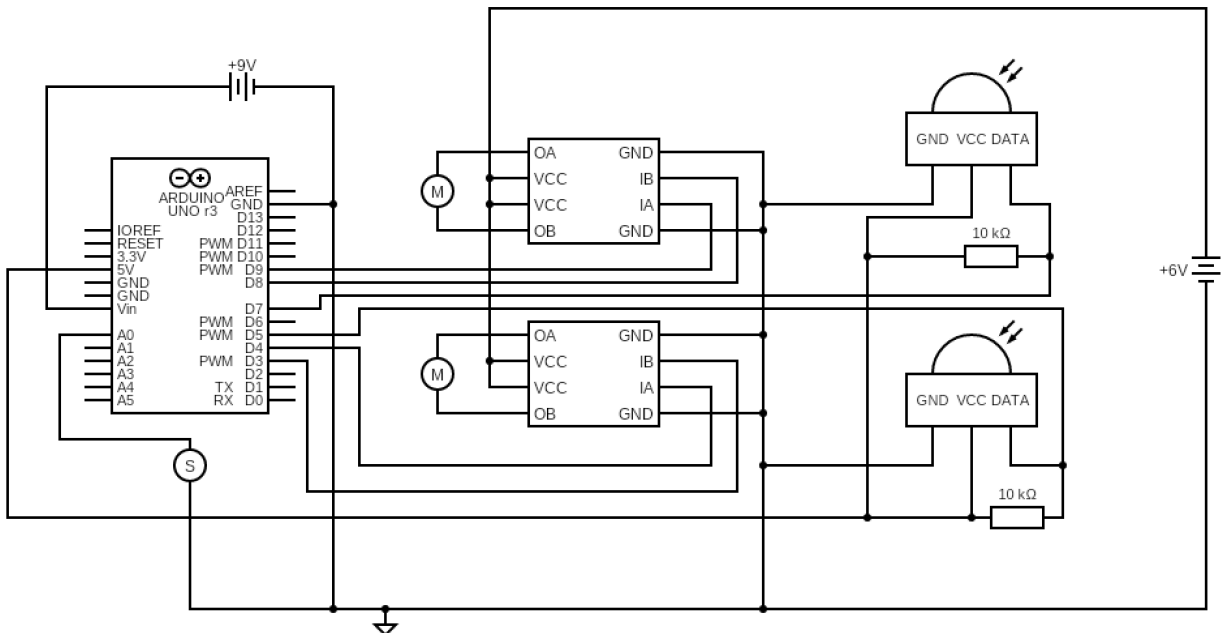
Even considering this, we did have some aspects that were very successful, specifically our QTI sensors. There was a moment of worry after the first set of matches happened and many robots ran off the board. We never tested our robot under the lighting conditions in Duffield atrium, but we had ours mounted low enough that they held strong and guided our robot to remain on the board for all the matches (unless pushed off by another robot).

Recommendations for future groups would definitely be to add a decent amount of weight to your robot to avoid it being pushed around. Additionally, prioritizing not just cube collection, but cube retention in your design, and to account for necessary components like the breadboard and batteries in your design.

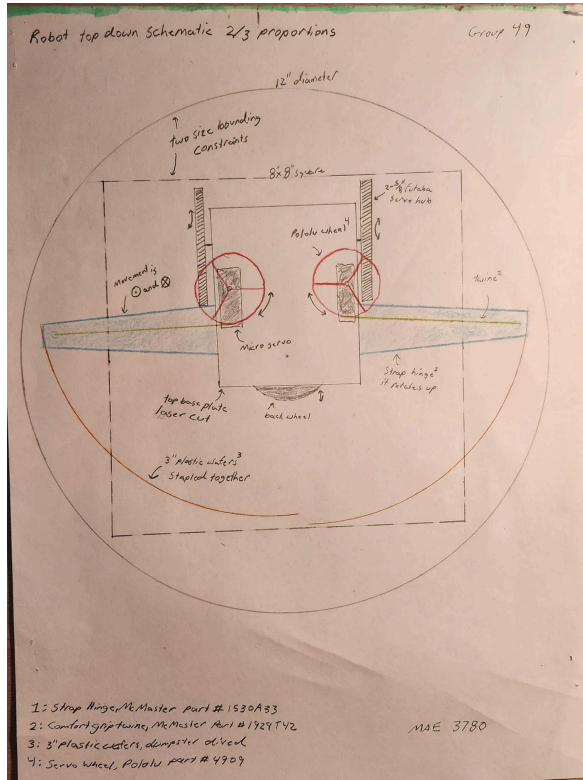
Appendix A (Bill of Materials)

Part Name	Part Number	Cost	QTY	Total Cost	Procurement Details
3" Single Wafer Shipper - Spring	H22-302-0615	\$1.64	100	\$1.64	(salvaged from Upson Hall free pile)
Pololu Wheel for Micro Servo Splines	4909	\$5.75	1	\$5.75	https://www.pololu.com/product/4909
Comfort-Grip Twine	1929T42	\$3.80	1	\$3.80	https://www.mcmaster.com/1929T42/
Strap Hinge	1530A33	\$2.81	2	\$5.62	https://www.mcmaster.com/1530A33/
1/2 oz. Wheel Weights	4991T115	\$3.09 (10 pack)	1	\$3.09	https://www.mcmaster.com/4991T115/
Acrylic Plate	—	\$6.83	1	\$6.83	RPL
				\$26.73	

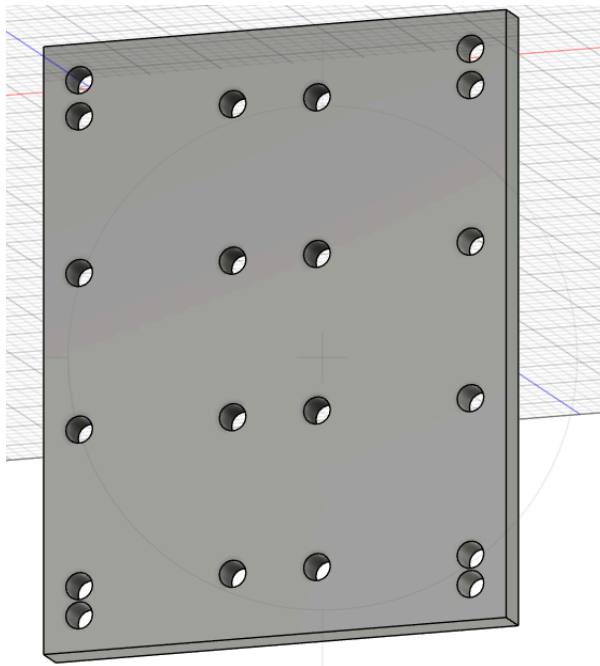
Appendix B (Circuit Diagram)



Appendix C (CAD Files and Drawings):



(Initial design sketch)



▼ Bounding Box	
Length	3.251 in
Width	4.188 in
Height	0.25 in
Center of Mass	1.375 in, -1.843 in, 0.12...

RPL Calculations:

Perimeter length = 14.878 in.

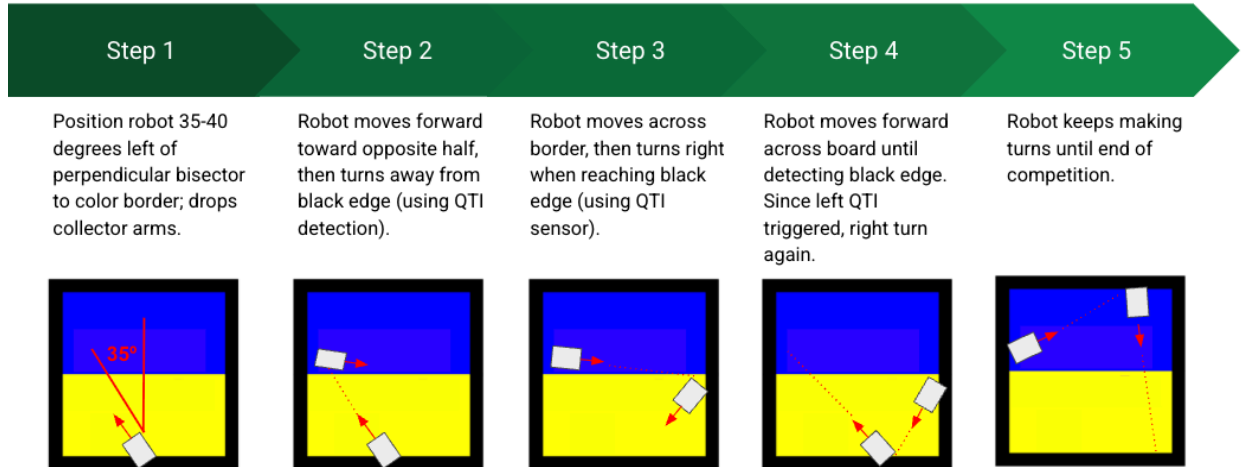
Hole circumferences: 0.589 in. Number of holes: 20.

Total cut length = 26.658in,

Total cost = \$5 (sheet) + \$1.33 (inch/cut) + \$0.50 (1part) = \$6.83

Appendix D (Flowchart)

Movement Flowchart



Appendix E: Code for Team 49

```
1  #include <avr/io.h>
2  #include <avr/interrupt.h>
3  #include <util/delay.h>
4  #include <stdint.h>
5
6  /* =====
7  SERVO
8  ===== */
9
10 void servo_init() {
11     DDRC |= 0b00000001; // set PC0 as output pin
12 }
13
14 void servo_pulse(uint16_t pulse_us) {
15     PORTC |= 0b00000001; // drive HIGH --> begin servo pulse
16
17     while (pulse_us--> 0) {
18         _delay_us(1);
19     }
20
21     PORTC &= 0b11111110; // drive LOW --> ends pulse
22     _delay_ms(18);
23 }
24
25 void servo_run() {
26     for (uint8_t i = 0; i < 10; i++) {
27         servo_pulse(1000);
28     }
29
30     for (uint8_t i = 0; i < 20; i++) {
31         servo_pulse(2000);
32     }
33 }
34
35 /* =====
36 MOTOR CONTROL
37 ===== */
38
39 void forward() {
40     PORTD |= 0b00001000; // D3 HIGH - left fwd
41     PORTB |= 0b00000010; // B1 HIGH - right fwd
42
43     PORTD &= 0b11101111; // D4 clear
44     PORTB &= 0b11111110; // B0 clear
45 }
46
47 void backward() {
48     PORTD |= 0b00010000; // D4 HIGH - left bwd
49     PORTB |= 0b00000001; // B0 HIGH - right bwd
50
51     PORTD &= 0b11110111; // D3 clear
52     PORTB &= 0b11111101; // B1 clear
53 }
54
55 void left_turn() {
56     PORTD |= 0b00010000; // D4 HIGH - left bwd
57     PORTB |= 0b00000010; // B1 HIGH - right fwd
58
59     PORTD &= 0b11110111; // D3 clear
60     PORTB &= 0b11111110; // B0 clear
```

```

61 }
62
63 void right_turn() {
64     PORTD |= 0b00001000; // D3 HIGH - left fwd
65     PORTB |= 0b00000001; // B0 HIGH - right bwd
66
67     PORTD &= 0b11101111; // D4 clear
68     PORTB &= 0b11111101; // B1 clear
69 }
70
71 void stop_motors() {
72     PORTD &= 0b11100111; // cut power left motor
73     PORTB &= 0b11111100; // cut power right motor
74 }
75
76 /* =====
77    QTI SENSORS
78    ===== */
79
80 void qti_init() {
81     DDRD  &= 0b01011111; // PD7 and PD5 as inputs
82     PORTD &= 0b01011111; // QTIs control pin input directly
83 }
84
85 uint8_t read_qti() {
86     uint8_t state = 0;
87
88     if (PIND & 0b10000000) { // if black (HIGH) detected on pin 7 (left QTI)...
89         state |= 0b00000010; // encoded detection of left QTI
90     }
91
92     if (PIND & 0b00100000) { // if black (HIGH) detected on pin 5 (right QTI)...
93         state |= 0b00000001; // encoded detection of right QTI
94     }
95
96     return state;
97 }
98
99 void escape_border(uint8_t state) { // what to do when black border detected...
100     stop_motors(); // stop moving
101
102     backward(); // back up 400 ms
103     _delay_ms(400);
104
105     stop_motors(); // stop moving
106     _delay_ms(50);
107
108     if (state == 0b00000010) { // if left QTI hit first (border is to left of robot)...
109         right_turn(); // turn right away from border
110         _delay_ms(540);
111
112     } else if (state == 0b00000001) { // if right QTI hit first (border is to right of robot)...
113         left_turn(); // turn left away from border
114         _delay_ms(540);
115
116     } else { // if simultaneous QTI detection (border is straight on or corner)...
117         right_turn(); // turn right away from border
118         _delay_ms(540);
119     }
120
121     stop_motors();
122     _delay_ms(50);
123 }
124

```



```

125  /* =====
126  FIND OPEN HEADING
127  ===== */
128
129  void find_open_heading() { // robot turns and scans - useful if robot gets cornered or pushed
    to black edge
130      uint8_t steps = 0;
131      int8_t dir = 1;
132
133      while (1) {
134          uint8_t qti = read_qti();
135
136          if (qti == 0) {
137              stop_motors();
138              return;
139          }
140
141          if (qti == 2) { // if left QTI detects, scan to the right
142              _delay_ms(30);
143              dir = 1;
144
145          } else if (qti == 1) { // if right QTI detects, scan left
146              _delay_ms(30);
147              dir = -1;
148
149          } else {
150              stop_motors();
151              _delay_ms(10);
152
153              backward();
154              _delay_ms(400);
155
156              stop_motors();
157          }
158
159          if (dir > 0) right_turn();
160          else left_turn();
161
162          _delay_ms(80);
163
164          stop_motors();
165          _delay_ms(20);
166
167          steps++;
168
169          if (steps >= 60) { // if no clear heading, scan in opposite direction
170              dir = -dir;
171              steps = 0;
172          }
173      }
174  }
175
176  /* =====
177  CORNER DETECTION
178  ===== */
179
180  uint8_t is_alternating(uint8_t *hist) {
181      for (uint8_t i = 1; i < 3; i++) { // checks if consecutive QTI detected values are caused by
    the same side
182          if (hist[i] == hist[i-1]) return 0;
183          if (hist[i] == 3) return 0;
184          if (hist[i-1] == 3) return 0;
185      }

```

```

186     return 1; // returns 1 if 3 consecutive QTI triggers are alternating sides (robot bouncing
between walls)
187 }
188
189 void escape_corner(uint8_t last) { // sequence to escape a corner
190     stop_motors();
191     _delay_ms(30);
192
193     backward(); // back up to clear walls
194     _delay_ms(500);
195
196     stop_motors();
197     _delay_ms(50);
198
199     if (last == 2) right_turn(); // if last side triggered was left, then turn right
200     else          left_turn();  // if last side triggered was right, then turn left
201
202     _delay_ms(700);
203
204     stop_motors();
205     _delay_ms(50);
206 }
207
208 /* =====
209 COMPETITION LOGIC
210 ===== */
211
212 uint8_t phase1_angled_cross() { // initial plan to traverse board on diagonal
213     forward();
214
215     while (1) {
216         _delay_ms(10);
217
218         uint8_t qti = read_qti(); // polls QTIs continuously to detect black border
219
220         if (qti == 0) continue; // if no border detected, continue forward
221
222         _delay_ms(20);
223
224         uint8_t confirm = read_qti(); // second reading of QTI
225
226         if (confirm != qti) continue;
227
228         if (confirm == 2 || confirm == 1) { // if right (1) or left (2) QTI triggered...
229             stop_motors();
230             return confirm;
231         }
232
233         escape_border(3); // run escape corner plan
234         find_open_heading();
235         forward();
236     }
237 }
238
239 void phase2_align_parallel(uint8_t first) { // game plan to turn parallel to blue-yellow
border
240     _delay_ms(100);
241
242     if (first == 1) left_turn(); // if started going to right, turn left parallel
243     else          right_turn(); // if started going to left, turn right parallel
244
245     _delay_ms(900);
246
247     stop_motors();

```

```

248     _delay_ms(100);
249 }
250
251 void phase3_roam() { // movement to roam around board after initial parallel traversal
252     uint8_t hist[3] = {0};
253     uint8_t idx = 0;
254     uint8_t count = 0;
255
256     forward();
257
258     while (1) {
259         _delay_ms(10);
260
261         uint8_t qti = read_qti();
262
263         if (qti != 0) {
264             _delay_ms(20);
265
266             uint8_t confirm = read_qti();
267
268             if (confirm == qti) {
269                 hist[idx] = qti;
270                 idx = (idx + 1) % 3;
271
272                 if (count < 3) count++;
273
274                 if (count == 3 && is_alternating(hist)) { // if the QTI alternates 3 times, corner
detected...
275                     escape_corner(confirm); // and run corner escape plan
276
277                     hist[0] = hist[1] = hist[2] = 0; // if QTI stops alternating, no corner has
been detected
278                     idx = 0; // and operate as normal
279                     count = 0;
280
281                     } else {
282                         escape_border(confirm);
283                     }
284
285                     find_open_heading();
286                     forward();
287                 }
288             }
289         }
290     }
291
292 void competition_run() { // executes diagonal cross, color parallel traversal, and then
roaming rest of match
293     find_open_heading();
294
295     uint8_t first = phase1_angled_cross();
296
297     phase2_align_parallel(first);
298     phase3_roam();
299 }
300
301 /* =====
302     MAIN
303     ===== */
304
305 int main(void) {
306     DDRB |= 0b00000011; // B0 and B1 as outputs for right motor
307     DDRD |= 0b00011000; // D3 and D4 as outputs for left motor
308

```

```
309     qti_init();           // initialize QTI sensors
310     servo_init();         // initialize servos
311
312     stop_motors();        // stop motors once reset button is hit
313
314     servo_run();          // run the servos to drop arms
315     competition_run();    // run the competition plan
316
317     while (1) {           // code to stop robot
318         stop_motors();
319     }
320
321     return 0;
322 }
```